

ArogyaMitra: An AI-Assisted Real-Time Telemedicine Consultation Platform

Project Proposal for UCS503P
Submitted to: Dr. Paramveer Kaur
Thapar Institute of Engineering and Technology

Animesh Sudhanshu - 1024170375 Ashish Bhagat - 1024170372
Devansh Rathaur - 1024170380

August 16, 2026

1 Higher-order goal

... secondary framing

This project aims to improve access to primary healthcare by reducing the friction between a patient needing medical advice and a doctor available to give it. While the broader motivation is healthcare equity (India's government telemedicine platform), the immediate engineering objective is to translate *low-friction, real-time doctor-patient consultation* into a measurable, deployable software solution built and iterated on by a small student team.

2 Time-to-value

... primary heuristic

- We will deliver meaningful increments quickly by cloning and adapting an existing open-source reference implementation, then prototyping the core consultation workflow and validating it within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations (and targeting CI-backed automation early) to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration (a working doctor-patient consultation loop), then improved based on user feedback.

3 Problem Statement

Patients, particularly those in smaller towns with limited specialist availability, face delays and friction when seeking timely medical advice for non-emergency conditions. Common pain points include:

- **Access friction:** booking an in-person consultation requires travel and waiting, even for simple queries.
- **Fragmented communication:** follow-up questions, prescriptions, and advice are often exchanged informally (calls, messaging apps) with no persistent record.
- **No triage before consultation:** patients cannot easily get a first-pass sense of urgency or relevant information before a doctor is available.

- **Scheduling opacity:** patients cannot easily see doctor availability or track the status of a booked consultation.

Why this matters now (contextual relevance): Real-time web technologies (WebRTC, WebSockets) and affordable managed backends have matured enough that a small team can build a credible consultation platform in an academic timeframe, while conversational AI can plausibly assist (not replace) the pre-consultation triage step.

4 Proposed Solution

... Software Proposition

4.1 Overview

Build a web-based “ArogyaMitra” real-time telemedicine platform

- **Patient portal:** registration, profile management, doctor discovery, and appointment booking.
- **Doctor dashboard:** availability management, appointment queue, and consultation history.
- **Real-time consultation:** secure real-time chat and video call between doctor and patient.
- **AI pre-consultation assistant:** a conversation-aware assistant that helps patients describe symptoms and prepare context before a doctor joins, with a safety layer that always defers emergency-flagged symptoms to immediate care.
- **Notifications:** real-time alerts when a consultation is about to begin or a doctor becomes available.

4.2 Core workflow

1. Patient registers/logs in and books a consultation slot with an available doctor.
2. Patient optionally interacts with the AI assistant to summarize symptoms ahead of time.
3. Both parties receive a real-time notification when the consultation is about to start.
4. Doctor and patient connect via real-time chat / video call; doctor records notes and advice.
5. Patient can review consultation history and past notes from their profile.

4.3 Operational constraints for an educational setting

- Keep the solution usable on low-to-mid range devices via responsive web design.
- Avoid dependence on novel research algorithms; focus on workflow correctness, real-time reliability, and safe AI-assistance boundaries.
- Design for deployability using free-tier or low-cost managed hosting suitable for a pilot (e.g., Render, Netlify, MongoDB Atlas).

5 Solution Approach

... Engineering focus

This is an engineering course project, so we focus on integration, workflow correctness, and safe deployment of existing well-understood technologies rather than novel algorithm research.

5.1 AI assistant scope

... non-research

The AI pre-consultation assistant is an integration task, not a research contribution:

- Use an existing LLM API (e.g., Gemini) for conversational symptom intake; no in-house model training.
- Add a lightweight safety/triage layer that detects emergency-flagged keywords (e.g., chest pain, stroke signs, self-harm) and always routes to an “seek immediate care” response rather than open-ended advice.
- The assistant only prepares context for the doctor; it never issues a diagnosis or prescription.

5.2 Web architecture

... web-based solution

A typical decoupled web architecture, matching the reference repository:

- **Frontend:** Next.js with a component library (Shadcn) for patient/doctor portals and the consultation UI.
- **Backend API:** Node.js/Express for authentication, appointment logic, and consultation record management.
- **Database:** MongoDB Atlas for users, appointments, consultation notes, and message history.
- **Real-time layer:** Socket.io for chat/notifications; WebRTC for video calls.
- **Authentication:** Firebase Authentication for patient and doctor accounts.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests (lint, unit) on every change via GitHub Actions.
- Produce repeatable deployment artifacts to a staging environment (Render for backend, Netlify for frontend).
- Enable safe and frequent releases of incremental features.

6 Evaluation Criterion

... measurable and attributable

We define success using measurable metrics tied to the core consultation workflow.

6.1 Primary evaluation metric

Time-to-connect (TTC): time between a scheduled consultation’s start time and the moment both doctor and patient are connected in the same chat/video session.

- Target: median $TTC \leq 2$ minutes during pilot testing.
- Attribution: measured from server-side timestamps (scheduled time vs. session-join event).

6.2 Secondary evaluation metrics

- **Booking completion rate:** percentage of started bookings that result in a confirmed appointment.
- **AI assistant safety accuracy:** percentage of emergency-flagged test cases correctly routed to “seek immediate care” during a fixed test suite run.
- **User clarity:** patient-reported clarity of the consultation flow using a simple 1–5 feedback question.
- **Reliability:** availability of staging/production for login, booking, and consultation during test windows (e.g., ≥ 99)

6.3 Pilot validation plan

... high-level

- Recruit 10–15 pilot patients and 3–5 volunteer/simulated doctor accounts.
- Run a fixed AI-assistant test suite (symptom prompts including emergency cases) before each release to catch safety regressions.
- Compare TTC and booking completion rate across iterations.

7 Scalability

... with foresight

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in number of concurrent consultations and users by:
 - decoupling frontend and backend (already true of the Next.js/Express split),
 - using stateless REST endpoints with Socket.io namespaces scoped per consultation room,
 - indexing appointment and user queries in MongoDB.
2. **Operationally deployable (practically):** The system should run on common managed hosting:
 - MongoDB Atlas as a managed database,
 - Render/Netlify as platform-hosted deployment targets,
 - CI/CD pipeline for staging/production via GitHub Actions.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is already available and proven in the reference repository, so the team focuses on workflow and safety correctness rather than infrastructure:

- Next.js for the frontend framework.
- Node.js/Express for REST API backend.
- MongoDB Atlas as the managed document database.
- Firebase Authentication for identity.

- Socket.io for real-time chat/notifications; WebRTC for video.
- Gemini API / Vercel AI SDK for the conversational assistant.
- GitHub Actions as the CI runner; Render/Netlify as deployment targets.

9 Project scope and deliverables

... iteration-friendly

9.1 Initial deliverable

... first iteration

- Fork/clone the reference repository and set up local + staging environments.
- Patient and doctor authentication and profile pages.
- Appointment booking flow with doctor availability.
- Real-time chat consultation between doctor and patient.
- Basic logging and timestamps for TTC measurement.
- CI: build + lint + backend unit tests.
- CD to staging (Render + Netlify) with a single pipeline job.

9.2 Subsequent deliverables

- Real-time video call integration (WebRTC) alongside chat.
- AI pre-consultation assistant with the emergency-triage safety layer.
- Real-time notifications for upcoming/available consultations.
- Admin/analytics view of bookings, TTC, and completion rate.
- Regression test suite for AI assistant safety behaviour.

10 Risks and mitigations

- **Data privacy / consent:** minimize collected medical data; store only what is required for the consultation record; never persist raw AI assistant conversations beyond what is needed for the doctor's context.
- **AI safety failure:** maintain a small, fixed regression test suite of emergency-symptom prompts that must pass before every release; the assistant never issues a diagnosis or prescription.
- **Real-time reliability:** use WebRTC/Socket.io connection-state handling and fallbacks (e.g., reconnect logic) so a dropped session does not lose consultation context.
- **Staff/doctor adoption:** keep the doctor dashboard minimal (availability toggle, queue, notes) to reduce onboarding friction for simulated/volunteer doctors during the pilot.
- **Operational issues in pilot:** use staging early; ensure CI/CD produces repeatable deployments before the pilot window.

11 Summary

This project proposes a deployable, AI-assisted real-time telemedicine platform, to reduce the friction between patients and doctors for non-emergency consultations. It meets the course criteria by emphasizing time-to-value through rapid iterations on a proven codebase, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially time-to-connect and AI assistant safety accuracy). It remains focused on engineering integration, workflow correctness, and safe AI-assistance boundaries rather than research-driven novelty.